

Verifiable Layered Architecture Visualizations

Invariants for Correct Level Computation

[Draft for co-author review]

Johannes Weigend

Weigend AM GmbH & Co.KG, Germany/Brazil
johannes.weigend@weigend.de

Veronika Schwarz

[Affiliation to be added]
veronika.schwarz@qaware.de

Michael Philippsen

Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
michael.philippsen@fau.de

Keywords: software architecture visualization, layout invariants, layered software city, strongly connected components, dependency analysis, software engineering tools

Abstract: Layered architecture visualizations communicate dependency structure through vertical position: elements that depend on others are placed higher. If the level-assignment pipeline is wrong, the picture can still look plausible while misleading the viewer: cyclic peer packages may be placed on different levels, or a regular dependency may point against the intended layer direction. Layout correctness should be the tool’s responsibility, not the reader’s — the implementation shows how hard that actually is.

Our 2021 IVAPP Paper, *A Layered Software City for Dependency Visualization* (Dashuber et al., 2021), introduced a city layout in which the X/Y position of class buildings follows dependency-derived levels, while building height along the Z axis encodes orthogonal metrics such as complexity or lines of code. S202 reimplements and validates the dependency-level pipeline of that layout as an open-source Java tool under Apache 2.0, together with a Layout Invariant Checker that verifies the layout after every analysis run.

Correct level computation turns out not to be a single ordering problem. It requires coordinating three inter-dependent constraint systems: class ordering, package containment, and level equality within SCCs. S202 therefore computes two semantically different level concepts: global architectural depth as a longest path in the SCC-collapsed class dependency DAG, and local layout position within each package, starting at zero and using only sibling dependencies. A subsequent horizontal row-ordering pass keeps same-row elements with dependencies close to each other without changing their levels. Highly cyclic code bases are handled by a heuristic SCC breaker; its cut edges are not hidden but marked as violations.

The Layout Invariant Checker verifies four postconditions: non-heuristic class dependencies must point downward (R1); packages in a filtered package-level SCC must share a level (R2); a container’s level must equal or exceed the maximum level of its contents (R3); and edge classifications must match the current level and SCC state (R5). A fifth rule, R4, was deliberately retired: its generalization of R2 to the unfiltered package graph produced false positives, because parent-child edges and heuristic back-edges created apparent cycles between packages that are not architectural peers (see Appendix A). The checker distinguishes implementation bugs from tolerated heuristic violations and emits a copy-paste-ready reproducer block for every finding.

We will demonstrate the checker on real code bases, tracing the path from a marked layout element to a failing reproducer test. In the 92-module *software-ekg-7* system, an R2 finding exposed a missing package-SCC equalization step and directly triggered a pipeline fix. All four invariants were ported to an independent Unity/.NET 3D software city and produced identical findings on shared inputs. The separation of global and local level concepts is not a design choice—no correct level-computation pipeline can collapse both questions into a single pass. To our knowledge, no previous layered architecture visualization has provided this degree of machine-checkable self-validation; we also discuss generalization to other layered visualization techniques.

REFERENCES

- Dashuber, V., Philippsen, M., and Weigend, J. (2021). A layered software city for dependency visualization. In *Proceedings of the 12th International Conference on Information Visualization Theory and Applications (IVAPP)*, VISIGRAPP 2021, pages 15–26. SCITEPRESS. Best Paper Award.
- Eades, P., Lin, X., and Smyth, W. F. (1993). A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.

Appendix A – Why Naive Single-Pass Does Not Work: Failure Modes

The Core Problem: Three Interacting Constraint Systems

A correct layered layout must satisfy three constraints at the same time:

1. **Class constraint (R1):** If $A \rightarrow B$ means that A depends on B , then $\text{level}(A) > \text{level}(B)$.
2. **Hierarchy constraint (R3):** A package must be placed at least as high as its highest contained element.
3. **SCC constraint (R2):** All classes in a cyclic dependency group receive the same level; packages that form a package-level SCC must also share a level.

Each constraint is individually manageable. Together they create a circular dependency system between the levels of computation themselves, not merely between classes.

Failure Mode 1: Cascading Level Updates

An algorithm iterates over classes and assigns levels. It first sees class A in package P_1 , computes $\text{level}(A) = 3$, and therefore sets $\text{level}(P_1) = 3$. Later it reaches class B in package P_2 and discovers that A and B are in the same SCC and must share a level, now $\text{level}(A) = \text{level}(B) = 5$. This forces P_1 to be recomputed. If P_1 depends on P_3 , P_3 may need to move as well. A cascade follows.

In a naive single-pass implementation, however, P_1 and P_3 may already have been finalized. SCC membership is only known after the relevant graph has been traversed; if level assignment and SCC discovery are interleaved, the iteration order decides whether the correction is still applied in time.

Failure Mode 2: Interleaved Cycle Detection Produces Classpath-Dependent Results

The most subtle practical failure mode occurs when cycle detection and level computation are not separated. The algorithm traverses the class graph, assigns levels, and handles discovered cycles on the fly, depending on which class happens to be processed first.

Classes come from JARs. The order in which the JVM sees classes follows the order in which they were packaged into the JAR. That order can depend on the build tool, module order in a multi-module build, filesystem ordering on the build server, and other implementation details. None of these orders is semantically meaningful; none encodes architectural dependency direction.

The result is unreliable: the same source code can produce different level layouts under different build configurations. The errors are subtle. The layout may still look plausible, arrows may mostly point in the expected direction, and no obvious visual break appears. Yet the precise hierarchy information is corrupted and may remain unnoticed for a long time.

The solution is to separate the phases strictly: first compute the SCC partition on the complete graph, then build the SCC-DAG, then assign levels. Tarjan's algorithm computes the SCC partition in $O(V + E)$; with deterministic input ordering, the implementation becomes reproducible and independent of classpath traversal artifacts.

Formal Counterexample: Why a Naive Recursive Algorithm Fails

The intuitive level-computation algorithm is a recursive DFS with memoization:

```
level(A) :  
  if A already computed: return level(A)  
  if A has no dependencies: return 0  
  return max(level(dep) for dep in A.dependencies) + 1
```

For acyclic graphs this is correct and equivalent to longest-path computation on a DAG. For cyclic graphs it fails structurally.

Counterexample with two SCCs:

$SCC_1 = \{A, B\}$, with $A \rightarrow B \rightarrow A$
 $SCC_2 = \{C, D\}$, with $C \rightarrow D \rightarrow C$
 Cross dependency: $A \rightarrow C$

The correct levels are $level(C) = level(D) = 0$ and $level(A) = level(B) = 1$.

Recursive algorithm, starting at A:

```

level(A)  [A "in progress", sentinel = 0]
-> level(B)
    -> level(A): cycle detected, return 0
    B.level = 0 + 1 = 1
-> level(C)  [C "in progress", sentinel = 0]
    -> level(D)
        -> level(C): cycle detected, return 0
        D.level = 0 + 1 = 1
    C.level = 1 + 1 = 2    <-- wrong; correct would be 0
A.level = max(1, 2) + 1 = 3    <-- wrong; correct would be 1

```

After SCC equalization, $SCC_1 = \max(3, 1) = 3$ and $SCC_2 = \max(2, 1) = 2$. Both results are wrong.

The cause is that the recursive algorithm enters SCC_2 from the context of SCC_1 . The sentinel-based local cycle handling inflates the level of SCC_2 , and that inflated level then feeds into the computation of SCC_1 . The correct level of SCC_2 , namely zero, is only known after the graph has been collapsed into SCCs and evaluated as an SCC-DAG.

Thus, recursive level algorithms that mix traversal, cycle handling, and level assignment can produce traversal-order-dependent results on graphs with interacting SCCs. In JVM-based tools, that traversal order can be influenced by classpath and packaging order. The correct solution collapses SCCs before level assignment: Tarjan, then SCC-DAG, then longest path.

Failure Mode 3: Mutual Dependency Between Package and Class Levels

Package P_1 is placed on the level of its highest class (R_3). But the level of that highest class depends on the classes it calls. Those classes live in packages whose levels are again derived from their contents and dependencies. The package graph is not available as a stable input beforehand; it is derived from the class graph.

This mutual dependency makes a shared single pass over classes and packages inherently unstable.

Failure Mode 4: The Big Lake – SCCs Without Heuristics Collapse the Layout

Without a mechanism for breaking large SCCs, all members of a cyclic group must be placed on the same level. For small cycles of two to five classes this is acceptable. For Minecraft Forge, however, we observed a single SCC containing 4,038 classes. Without a breaker, more than 40 percent of the project would land on one flat level at the bottom of the layout. Everything else would stand above it, but the interesting internal structure of the core would be invisible.

The heuristic SCC breaker solves this practical problem. It identifies back edges, meaning dependencies that run against the natural architectural flow, and temporarily removes them for level computation. The removed edges are visualized as violations instead of being ignored. The layout remains honest about architectural debt while still producing a usable hierarchy.

Relation to Feedback Arc Set Heuristics

Failure Mode 4 is an instance of the Minimum Feedback Arc Set problem: remove a minimal set of directed edges so the graph becomes acyclic. The problem is NP-hard (Karp, 1972), so any practical polynomial-time implementation must use a heuristic or approximation.

A well-known related heuristic is due to Eades, Lin, and Smyth (Eades et al., 1993). It orders nodes according to the difference between outgoing and incoming degree, then classifies edges that point backward in the resulting sequence as feedback edges. The intuition is similar to S202: high out-degree suggests a higher-level element; high in-degree suggests a foundational element.

S202 uses a deliberately simpler variant: a direct normalized score per node,

$$\frac{\text{outDegree} - \text{inDegree}}{\max(1, \text{outDegree} + \text{inDegree})}.$$

Edges from lower-ranked to higher-ranked nodes are classified as back-edge candidates. This is easier to implement and sufficient for software dependency graphs, where the heuristic reflects architectural intuition rather than claiming mathematical optimality.

The important distinction is transparency. Many tools break cycles implicitly, but S202 classifies heuristic cut edges explicitly. They are marked as `VIOLATION`, rendered in red, and known to the invariant checker. This makes algorithmic defects separable from tolerated heuristic artifacts.

The Solution: Two Semantically Different Level Concepts

Failure modes 1 to 3 share the same root cause: trying to use a single level concept for two different questions. S202 separates them:

- **Phase 1** computes architectural depth: the longest path in the global, cycle-free SCC-DAG across all classes of the analyzed project.
- **Phase 2** computes local layout position inside a package: a zero-based ordering derived only from sibling dependencies.

Both phases use the same conceptual algorithmic shape, but on different graphs with different constraints:

Algorithm 1: Phase 1 – global

```
class graph
  → Tarjan (SCCs) → SCC-DAG
  → longest-path pass → global levels
  → assign levels to SCC members
  → package-level propagation upward
```

Algorithm 2: Phase 2 – local, per package

```
sibling graph (internal edges only)
  → Tarjan (local SCCs) → local SCC-DAG
  → topological pass → local layout positions
```

Phase 1 is reproducible because SCCs are collapsed before levels are assigned. The SCC-DAG is acyclic by construction, and longest-path computation on that DAG is stable for the same graph.

Phase 2 intentionally ignores external dependencies. If package `ui` depends on `domain` and `domain` depends on `persistence`, external edges would otherwise pull many siblings into global levels and make their local structure unreadable. Phase 2 sees only sibling relationships and produces a package-local ordering that is independent of the global architectural depth.

No single level concept can serve both purposes well. A purely global level is often visually useless inside a package; a purely local level has no architectural meaning. The explicit separation enables a layout that is both structurally correct and readable.

	Phase 1	Phase 2
Meaning	Architectural depth in SCC-DAG	Visual position among siblings
Scope	Global — all project classes	Local — children of one package
Zero point	Leaf classes = 0; grows with depth	Always 0 for the lowest sibling
Cycles	Eliminated by SCC collapse before level assignment	Handled by local Tarjan
Value range	Project-dependent; can reach hundreds	Small integers (sibling count – 1)

Table 1: The two level concepts computed by S202.

Layout Invariants for Level Correctness

The two level phases define four machine-checkable postconditions verified by the Layout Invariant Checker after every analysis run:

Rule	Postcondition
R1	Non-heuristic class dependencies point strictly downward: $A \rightarrow B \Rightarrow \text{level}(A) > \text{level}(B)$
R2	All packages in a filtered package-level SCC share the same level
R3	A container’s level equals or exceeds the maximum level of its contents
R5	Edge classifications (NORMAL, VIOLATION, BACK_EDGE) are consistent with the current level assignment and SCC membership

Table 2: The four layout invariants verified after every analysis run.

When we deployed the invariant checker against large real-world code bases—including Minecraft Forge and *software-ekg-7*—it immediately surfaced R2 violations: subtle structural bugs that had gone completely unnoticed despite a fully green unit-test suite. These findings were only detectable through automated structural analysis; the checker’s own 156-test suite was fully green throughout. Figure 1 shows a sample R2 report from Minecraft Forge; the same checker exposed a real R2 bug in *software-ekg-7* (see also below).

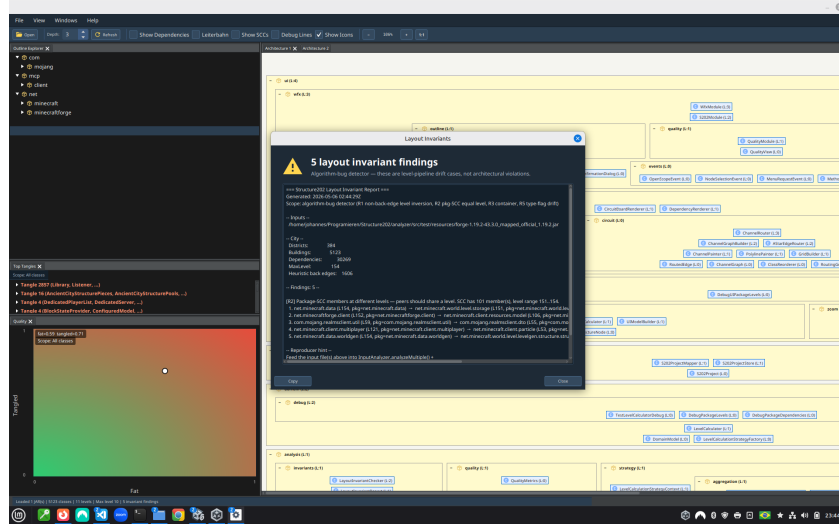


Figure 1: R2 checker report: packages in a filtered package-SCC are placed on different levels. Findings are reported as algorithm-bug candidates with a copy-paste-ready reproducer block, which makes the visual failure traceable to concrete level-pipeline constraints.

The invariant checker connects the visual symptom to a reproducible algorithm defect. The R2 finding shown in Figure 1 is precisely the class of defect addressed by the fixpoint in Step 5c, which is described in detail in the Implementation section below.

The Remaining Rules: R1, R3, and R5

R1 ($A \rightarrow B \Rightarrow \text{level}(A) > \text{level}(B)$) is the most straightforward invariant: it is a direct corollary of longest-path computation on the SCC-DAG and holds trivially for any correctly implemented level assignment. Its value lies not in detecting subtle bugs but in providing an immediate sanity check—a failing R1 signals a fundamental breakdown in the pipeline.

R3 (a container’s level $\geq \max$ level of its contents) is less obvious. Packages are assigned levels independently of the class-level phase; if the two phases are not composed in the right order, or if an incremental update touches only one of them, a package can end up *below* its highest-level class. The checker catches this silently drifting inconsistency that would otherwise only manifest as a visual anomaly in the city view.

R5 validates that every edge's stored classification (`NORMAL`, `VIOLATION`, `BACK_EDGE`) is still consistent with the current level assignment and SCC membership. Edge labels are computed during analysis and cached in the model; after any algorithmic change they can silently become stale. **R5** is therefore a cross-cutting consistency guard: it ties the structural output of the level pipeline back to the semantic annotations visible to the user.

R4: A Retired Rule and What It Teaches

An early version of the invariant checker contained **R4**:

`Cyclic child packages must share a level.`

The rule sounds reasonable and is correct in simple cases. In practice it failed because the raw package graph contains relationships that should not be interpreted as peer-level architectural cycles:

- **Parent-child edges** create apparent bidirectional relationships because a parent contains its children and children belong to their parent. Without filtering, many parent/child pairs look cyclic.
- **SCCBreaker back edges** connect packages that are not necessarily architectural peers. A rule that includes these edges equalizes the wrong candidates.
- **Shared class SCCs** can make two packages appear package-dependent without representing a true structural peer relationship.

R4 was replaced by **R2**, which expresses the same intention on the correct filtered graph. **R2** removes heuristic back edges, intra-subtree edges, and edges caused by shared class SCCs before running Tarjan on the remaining package graph.

Specifying invariants for a three-level constraint system turns out to be harder than it appears. The checker therefore provides a second safety net: it not only tests the implementation against the rules, but also exposes when the rules themselves are too broad or too coarse. False positives force the rule to become more precise. The later **R2** finding in *software-ekg-7* shows that the refined rule was precise enough to distinguish real pipeline bugs from apparent ones.

Implementation: The Complete Pipeline

The conceptual two-phase structure expands into the following concrete implementation steps plus a rendering-order optimization.

Algorithm 1: Phase 1 – Global Level Computation (`LevelCalculator`)

- Step 1:** Create class objects (all levels $\leftarrow 0$)
 - Step 2:** Create package objects (all levels $\leftarrow 0$)
 - Step 3:** Compute class levels
 - Tarjan on class graph
 - SCCBreaker: identify heuristic back edges in large SCCs ($|SCC| \geq 3$)
 - Remove back edges → filtered class DAG
 - Condense filtered DAG into SCC super-nodes
 - Single longest-path pass on condensed DAG
 - Assign SCC level to all members (equalization implicit)
 - Step 4:** Package level $\leftarrow \max(\text{level of contained classes})$
 - Step 5a:** Lift parent package levels — initial bottom-up pass:
sort packages deepest-first; parent.level $\leftarrow \max(\text{child.level})$
 - Step 5b:** Order packages by cross-package dependencies — DFS post-order pass:
DFS post-order on simple cross-package graph; pkg.level $\leftarrow \max(\text{dep.level}) + 1$
 - Step 5c:** Equalize package SCCs (R2) and re-lift parents to a fixpoint:
repeat Tarjan-SCC max equalization and parent lift until no package level changes
 - Step 6:** Set reverse dependencies (dependents)
-

After Step 3, level is no longer a property of an individual class alone. It is the level of the SCC node to which the class belongs, measured as the longest path in the cycle-free quotient graph. That graph covers all classes of the analyzed project, across packages. Cycles do not influence level assignment internally because they have already been collapsed before longest-path computation. This is the fundamental difference from naive recursive approaches, and it is the reason why Phase 2 must be a separate computation.

Algorithm 2: Phase 2 – Local Visualization Positions (`DistrictRowLevelCalculator`, recursively per package)

- for each** package P :
 - Step 1:** Collect subtree classes per sibling
(class: {itself}; package: all classes recursively)
 - Step 2:** Build sibling graph via distinct-target counting:
 - count $_{A \rightarrow B} = |\{c \in \text{subtree}(B) \mid c \in \text{deps}(A)\}|$
 - count $_{B \rightarrow A} = |\{c \in \text{subtree}(A) \mid c \in \text{deps}(B)\}|$
 - add edge $A \rightarrow B$ **if** count $_{A \rightarrow B} > \text{count}_{B \rightarrow A}$
 - Step 3:** Tarjan → local SCC-DAG → longest-path → local levels
 - Step 4:** Apply levels to all siblings
 - Step 5:** Recurse into child packages
-

Step 5c is intentionally a monotone fixpoint, not an arbitrary bounded loop. The R2 equalization step can lift a package-SCC member to the maximum level of its peer group. A subsequent parent lift can then raise a parent package that is itself part of another package-SCC, which may require one more R2 equalization. Both operations only raise levels and only copy existing maximum levels; no rule creates an unbounded sequence of fresh levels. Since the number of packages and level values is finite, the iteration terminates when neither operation changes any package level. The result is the least fixpoint satisfying package-SCC equality (R2) and container dominance (R3) after cross-package ordering.

A single-pass implementation is possible, but only by moving complexity earlier: one must first build a combined constraint graph containing the filtered package-dependency constraints and the child-to-parent con-

tainment constraints, condense that graph into components, initialize each component with the maximum level of its members, and then run a single topological max-propagation pass on the resulting component DAG. The implemented fixpoint keeps these concerns explicit and is validated by the same R2/R3 invariants.

The term “two phases” refers to the conceptual separation between global architectural depth and local layout position, not to the number of implementation steps. Phase 1 is an ordered pipeline with one monotone package fixpoint. Phase 2 is an independent local computation. Phase 3 is a same-row ordering optimization over the already fixed Phase 2 rows.

Algorithm 3: Phase 3 – Horizontal Row Ordering (`HorizontalRowLayoutOptimizer`)

```

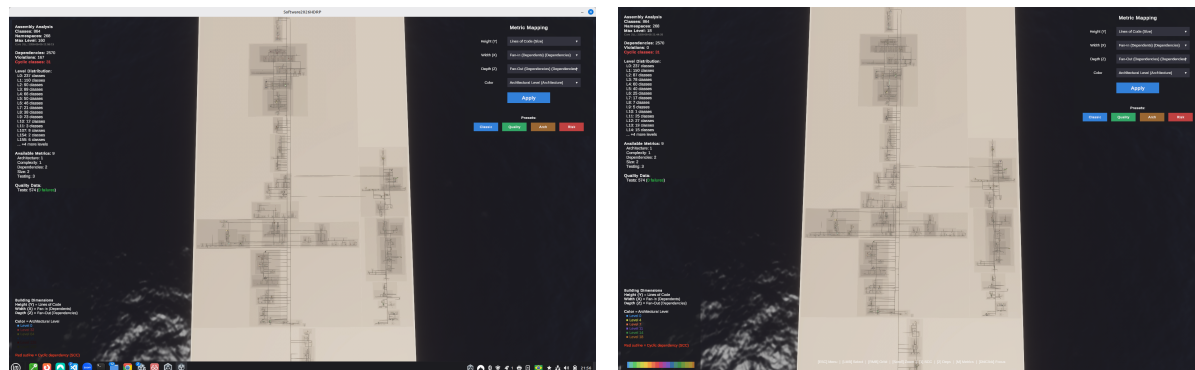
for each package P:
  Step 1: Group direct children by their Phase 2 level
  Step 2: Collect subtree classes per row element
  Step 3: Build weighted undirected proximity links from inter-sibling class dependencies
  Step 4: Run bounded barycentric sweeps across rows
  Step 5: Store the result in horizontalLayoutOrder
rendering: sort a copied child view by level, then horizontalLayoutOrder, then full name

```

Phase 3 is not a third level concept. It does not move an element to another row and is deliberately excluded from the layout invariants. Its only purpose is to make optional dependency overlays more legible: if dependencies are drawn, same-row elements connected to nearby rows should appear horizontally near each other. The implementation stores this as a separate `horizontalLayoutOrder` field and never sorts the semantic child list.

R2 Fix in *software-ekg-7*: Visual Evidence

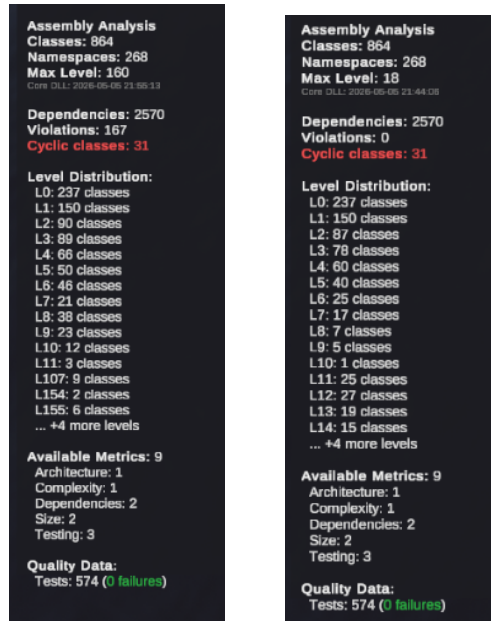
The *software-ekg-7* case is the concrete failure that forced the monotone Step 5c fixpoint. Before the fix, the city view still looked plausible: the package rectangles and dependency streets formed a readable layout, and no single visual element made the pipeline defect obvious. The metrics panel, however, exposed the inconsistency: 167 dependency violations were reported and the maximum architectural level had inflated to 160. After the R2 fixpoint was introduced, the same input produced zero violations and a maximum level of 18. The number of classes, packages, dependencies, cyclic classes, and tests stayed constant; the change is therefore attributable to level propagation, not to a different input model.



(a) Before the R2 fix: the full city view looks plausible, but the metric panel reports 167 violations and MaxLevel 160.

(b) After the monotone R2 fixpoint: the same analyzed system reports zero violations and MaxLevel 18.

Figure 2: Full-screen before/after screenshots of the *software-ekg-7* software city. The visible city changes only subtly; the level metrics reveal the correction.



(a) Before: 167 violations, MaxLevel 160.

(b) After: zero violations, MaxLevel 18.

Figure 3: Cropped metric panels from Figure 2. These panels are the clearest user-visible symptom of the R2 propagation defect and its fix.

Summary

Correct level computation for layered architecture visualizations requires satisfying three interacting constraint systems simultaneously: class dependency ordering, package containment (R3), and SCC level equality (R2). No single traversal pass can satisfy all three; the failure modes documented above are structural, not accidental.

S202 resolves this by separating two semantically distinct level concepts: global architectural depth, computed as the longest path in the SCC-collapsed class dependency DAG, and local layout position within a package, derived exclusively from sibling dependencies. A separate horizontal row-ordering pass improves dependency overlays without changing those levels or mutating the semantic child order. The Layout Invariant Checker verifies four machine-checkable postconditions (R1, R2, R3, R5) after every analysis run and emits copy-paste-ready reproducer blocks for each finding.

The *software-ekg-7* case demonstrates that the checker catches defects invisible to visual inspection: a plausible city layout reported 167 violations and a maximum architectural level of 160; after the R2 equalization step was corrected, the same input produced zero violations and a maximum level of 18, with no change to the analyzed source code.

Appendix B – LevelCalculator Reference Implementation

The following source excerpt is the complete Java implementation of the global level pipeline discussed above. It includes the monotone Step 5c fixpoint that replaced the former hard-coded iteration bound.

```
1 package de.weigend.s202.domain;
2
3 import de.weigend.s202.analysis.scc.SCCBreaker;
4 import de.weigend.s202.analysis.scc.StronglyConnectedComponent;
5 import de.weigend.s202.analysis.scc.TarjanSCCFinder;
6 import de.weigend.s202.analysis.strategy.LevelCalculationStrategyContext;
7 import de.weigend.s202.reader.DependencyModel;
8
9 import java.util.*;
10 import java.util.logging.Logger;
11
12 /**
13  * Calculates architectural levels for classes and packages based on dependencies.
14  *
15  * Phase 1 pipeline:
16  *   Step 1 Create class objects (all levels = 0)
17  *   Step 2 Create package objects (all levels = 0)
18  *   Step 3 Compute class levels Tarjan → SCCBreaker (iterative) → SCC-DAG → longest-path
19  *   Step 4 Package level = max(contained class levels)
20  *   Step 5a Lift parent package levels single bottom-up pass (deepest-first sort)
21  *   Step 5b Cross-package dependency order DFS post-order on simple cross-package graph
22  *   Step 5c Equalize package SCCs (R2) Tarjan on filtered graph → lift to max
23  *   Step 6 Set reverse dependencies
24  *
25  * Step 5b uses a simple cross-package graph (intra-subtree edges removed only) for
26  * one DFS post-order ordering pass. Step 5c uses the fully filtered graph
27  * (back-edges and shared-class-SCC edges also removed) to equalize only genuine cyclic
28  * peer packages -- the same graph the R2 invariant checker uses.
29  *
30  * Step 5c and the parent lift are coupled: SCC equalization can lift a child package,
31  * and parent lifting can then lift an SCC member again. They therefore run to a monotonic
32  * fixpoint: the loop stops when neither operation raises any package level.
33  */
34 public class LevelCalculator {
35
36     private static final Logger LOG = Logger.getLogger(LevelCalculator.class.getName());
37
38     private final LevelCalculationStrategyContext strategyContext;
39
40     public LevelCalculator() {
41         this(LevelCalculationStrategyFactory.createDefault());
42     }
43
44     public LevelCalculator(LevelCalculationStrategyContext strategyContext) {
45         Objects.requireNonNull(strategyContext, "strategyContext cannot be null");
46         this.strategyContext = strategyContext;
47     }
48
49     public DomainModel calculate(DependencyModel rawModel) {
50         DomainModel model = new DomainModel();
51
52         // Step 1: Create class objects (all levels = 0)
53         for (String className : rawModel.getAllClassNames()) {
54             DependencyModel.ClassInfo rawClass = rawModel.getClass(className);
55             model.addClass(className, new DomainModel.CalculatedElementInfo(
56                 className, rawClass.simpleName, "CLASS", 0,
57                 new HashSet<>(rawClass.dependencies), rawClass.interfaceType));
58         }
59     }
60 }
```

```

58     }
59
60     // Step 2: Create package objects (all levels = 0)
61     for (String packageName : rawModel.getAllPackageNames()) {
62         DependencyModel.PackageInfo rawPkg = rawModel.getPackage(packageName);
63         model.addPackage(packageName, new DomainModel.CalculatedElementInfo(
64             packageName, rawPkg.simpleName, "PACKAGE", 0, new HashSet<>());
65     }
66
67     // Step 3: Compute class levels (iterative SCCBreaker → SCC-DAG → longest-path)
68     calculateClassLevels(model);
69
70     // Step 4: Package level = max(level of contained classes)
71     setPackageLevelsFromClasses(model);
72
73     // Pre-compute both package graphs once -- Step 5b and 5c use different graphs.
74     Map<String, Set<String>> simplePkgGraph = buildSimplePackageGraph(model);
75     Map<String, Set<String>> filteredPkgGraph = buildFilteredPackageGraph(model);
76
77     // Step 5a: Lift parent package levels -- single bottom-up pass (deepest-first)
78     liftParentPackageLevels(model, rawModel);
79
80     // Step 5b: Cross-package dependency order -- single DFS post-order pass.
81     // Uses the simple cross-package graph (intra-subtree edges removed only).
82     // Cycles in this graph are handled gracefully by the DFS (in-stack skip);
83     // the resulting levels may be approximate for cyclic packages, which Step 5c corrects.
84     applyPackageDependencyOrdering(model, simplePkgGraph);
85
86     // Steps 5c + lift: equalize package SCCs (R2) then re-propagate to parents.
87     // The fixpoint is reached only when neither operation lifts any package.
88     // Both operations are monotonic and only copy existing maximum levels, so
89     // the loop terminates without an arbitrary iteration cap.
90     boolean changed;
91     do {
92         changed = equalizePackageSccLevels(model, filteredPkgGraph);
93         changed |= liftParentPackageLevels(model, rawModel);
94     } while (changed);
95
96     // Step 6: Set reverse dependencies
97     updateDependentRelationships(model);
98
99     return model;
100 }
101
102 public LevelCalculationStrategyContext getStrategyContext() {
103     return strategyContext;
104 }
105
106 // -----
107 // Step 3
108 // -----
109
110 private void calculateClassLevels(DomainModel model) {
111     Map<String, Set<String>> classDeps = new HashMap<>();
112     for (DomainModel.CalculatedElementInfo c : model.getAllClasses().values()) {
113         classDeps.put(c.fullName, new HashSet<>(c.dependencies));
114     }
115     Map<String, Integer> levels =
116         strategyContext.getClassLevelStrategy().calculateClassLevels(classDeps);
117     for (Map.Entry<String, Integer> e : levels.entrySet()) {
118         DomainModel.CalculatedElementInfo info = model.getClass(e.getKey());
119         if (info != null) info.setLevel(e.getValue());
120     }
121 }

```

```

120     }
121 }
122
123 // -----
124 // Step 4
125 // -----
126
127 private void setPackageLevelsFromClasses(DomainModel model) {
128     Map<String, Integer> max = new HashMap<>();
129     for (DomainModel.CalculatedElementInfo c : model.getAllClasses().values()) {
130         String pkg = extractPackageName(c.fullName);
131         if (pkg != null) max.merge(pkg, c.level, Math::max);
132     }
133     for (Map.Entry<String, Integer> e : max.entrySet()) {
134         DomainModel.CalculatedElementInfo info = model.getPackage(e.getKey());
135         if (info != null) info.setLevel(e.getValue());
136     }
137 }
138
139 // -----
140 // Step 5a -- single bottom-up pass
141 // -----
142
143 /**
144  * Lifts each parent package to the maximum level of its direct children.
145  * Packages sorted deepest-first (most dots) guarantee children are finalized
146  * before their parent is visited -- single pass, no loop needed.
147  *
148  * @return true if at least one package level was raised
149  */
150 private boolean liftParentPackageLevels(DomainModel model, DependencyModel rawModel) {
151     List<String> pkgNames = new ArrayList<>(model.getAllPackages().keySet());
152     pkgNames.sort((a, b) -> packageDepth(b) - packageDepth(a));
153
154     Map<String, DomainModel.CalculatedElementInfo> packages = model.getAllPackages();
155     boolean changed = false;
156     for (String pkgName : pkgNames) {
157         DependencyModel.PackageInfo rawPkg = rawModel.getPackage(pkgName);
158         if (rawPkg == null || rawPkg.childPackages.isEmpty()) continue;
159         DomainModel.CalculatedElementInfo pkgInfo = packages.get(pkgName);
160         if (pkgInfo == null) continue;
161         int maxChild = pkgInfo.level;
162         for (String child : rawPkg.childPackages) {
163             DomainModel.CalculatedElementInfo childInfo = packages.get(child);
164             if (childInfo != null && childInfo.level > maxChild) maxChild = childInfo.level;
165         }
166         if (maxChild > pkgInfo.level) {
167             pkgInfo.setLevel(maxChild);
168             changed = true;
169         }
170     }
171     return changed;
172 }
173
174 private static int packageDepth(String name) {
175     int d = 0;
176     for (int i = 0; i < name.length(); i++) if (name.charAt(i) == '.') d++;
177     return d;
178 }
179
180 // -----
181 // Step 5b -- single DFS post-order pass on simple cross-package graph

```

```

182 // -----
183
184 /**
185  * Applies cross-package dependency ordering via DFS post-order.
186  * Uses the simple package graph (intra-subtree edges removed only) so that the
187  * pass is  $O(V+E)$ . Cycles are skipped by the DFS (in-stack detection); any
188  * remaining level inconsistencies for cyclic packages are corrected in Step 5c.
189  */
190 private void applyPackageDependencyOrdering(DomainModel model,
191                                             Map<String, Set<String>> pkgGraph) {
192     if (pkgGraph.isEmpty()) return;
193     Map<String, DomainModel.CalculatedElementInfo> packages = model.getAllPackages();
194
195     for (String pkg : topoSortDfsPostOrder(pkgGraph)) {
196         DomainModel.CalculatedElementInfo pkgInfo = packages.get(pkg);
197         if (pkgInfo == null) continue;
198         for (String dep : pkgGraph.getOrDefault(pkg, Collections.emptySet())) {
199             DomainModel.CalculatedElementInfo depInfo = packages.get(dep);
200             if (depInfo != null && pkgInfo.level <= depInfo.level) {
201                 pkgInfo.setLevel(depInfo.level + 1);
202             }
203         }
204     }
205 }
206
207 /**
208  * Iterative DFS post-order traversal (dependencies before dependents).
209  * Cycles are handled by skipping nodes already on the current DFS stack.
210  */
211 private static List<String> topoSortDfsPostOrder(Map<String, Set<String>> graph) {
212     Set<String> visited = new HashSet<>();
213     Set<String> inStack = new HashSet<>();
214     List<String> result = new ArrayList<>(graph.size());
215
216     for (String start : graph.keySet()) {
217         if (visited.contains(start)) continue;
218
219         // Explicit stack of (node, dep-iterator) pairs -- avoids recursion depth limits.
220         Deque<Object[]> stack = new ArrayDeque<>();
221         Iterator<String> startDeps = graph.getOrDefault(start, Collections.emptySet()).iterator();
222         stack.push(new Object[]{start, startDeps});
223         inStack.add(start);
224
225         while (!stack.isEmpty()) {
226             Object[] frame = stack.peek();
227             String node = (String) frame[0];
228             @SuppressWarnings("unchecked")
229             Iterator<String> deps = (Iterator<String>) frame[1];
230
231             boolean pushed = false;
232             while (deps.hasNext()) {
233                 String dep = deps.next();
234                 if (!visited.contains(dep) && !inStack.contains(dep)) {
235                     Iterator<String> depDeps =
236                         graph.getOrDefault(dep, Collections.emptySet()).iterator();
237                     stack.push(new Object[]{dep, depDeps});
238                     inStack.add(dep);
239                     pushed = true;
240                     break;
241                 }
242             }
243
244             // Pop the node and its dependencies from the stack
245             stack.pop();
246             inStack.remove(node);
247             result.add(node);
248             visited.add(node);
249         }
250     }
251 }

```

```

244         if (!pushed) {
245             stack.pop();
246             inStack.remove(node);
247             visited.add(node);
248             result.add(node);
249         }
250     }
251 }
252 return result;
253 }
254
255 // -----
256 // Step 5c -- single Tarjan pass for R2
257 // -----
258
259 /**
260  * Equalizes every multi-member package-SCC to its members' maximum level.
261  * Uses the filtered graph (back-edges and shared-class-SCC edges removed) so
262  * that only genuine cyclic peer packages are equalized -- consistent with R2.
263  *
264  * @return true if any level was changed (expected when cycles exist only in
265  *         the simple graph used by Step 5b; logged at FINE level)
266  */
267 private boolean equalizePackageSccLevels(DomainModel model,
268                                         Map<String, Set<String>> filteredPkgGraph) {
269     if (filteredPkgGraph.isEmpty()) return false;
270     Map<String, DomainModel.CalculatedElementInfo> packages = model.getAllPackages();
271     boolean changed = false;
272     for (StronglyConnectedComponent scc : new TarjanSCCFinder(filteredPkgGraph).findSCCs()) {
273         if (scc.getSize() <= 1) continue;
274         int maxLevel = Integer.MIN_VALUE;
275         for (String m : scc.getMembers()) {
276             DomainModel.CalculatedElementInfo p = packages.get(m);
277             if (p != null && p.level > maxLevel) maxLevel = p.level;
278         }
279         if (maxLevel == Integer.MIN_VALUE) continue;
280         for (String m : scc.getMembers()) {
281             DomainModel.CalculatedElementInfo p = packages.get(m);
282             if (p != null && p.level < maxLevel) { p.setLevel(maxLevel); changed = true; }
283         }
284     }
285     return changed;
286 }
287
288 // -----
289 // Package graph builders
290 // -----
291
292 /**
293  * Simple cross-package graph: removes only intra-subtree edges (parent↔child).
294  * Used by Step 5b for the topo-sort ordering pass.
295  */
296 private Map<String, Set<String>> buildSimplePackageGraph(DomainModel model) {
297     Map<String, Set<String>> pkgGraph = new HashMap<>();
298     for (String pkg : model.getAllPackages().keySet()) pkgGraph.put(pkg, new HashSet<>());
299     for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
300         String fromPkg = extractPackageName(cls.fullName);
301         if (fromPkg == null) continue;
302         for (String dep : cls.dependencies) {
303             String toPkg = extractPackageName(dep);
304             if (toPkg == null || fromPkg.equals(toPkg)) continue;
305             if (isInSameSubtree(fromPkg, toPkg)) continue;

```

```

306         pkgGraph.get(fromPkg).add(toPkg);
307     }
308 }
309 return pkgGraph;
310 }
311
312 /**
313  * Filtered cross-package graph: additionally removes heuristic back-edges and
314  * edges caused by shared class SCCs. Used by Step 5c (R2 equalization) -- the
315  * same filtering the LayoutInvariantChecker applies for its R2 check.
316  */
317 private Map<String, Set<String>> buildFilteredPackageGraph(DomainModel model) {
318     Map<String, DomainModel.CalculatedElementInfo> classes = model.getAllClasses();
319     if (classes.isEmpty()) return Collections.emptyMap();
320
321     Map<String, Set<String>> classGraph = new HashMap<>(classes.size());
322     for (DomainModel.CalculatedElementInfo cls : classes.values()) {
323         Set<String> deps = new HashSet<>();
324         for (String d : cls.dependencies) if (classes.containsKey(d)) deps.add(d);
325         classGraph.put(cls.fullName, deps);
326     }
327
328     Map<String, Integer> classToSccId = new HashMap<>();
329     for (StronglyConnectedComponent scc : new TarjanSCCFinder(classGraph).findSCCs()) {
330         for (String m : scc.getMembers()) classToSccId.put(m, scc.getId());
331     }
332     Set<SCCBreaker.Edge> backEdges = new SCCBreaker(classGraph).findBackEdges();
333
334     Map<String, Set<String>> pkgGraph = new HashMap<>();
335     for (String pkg : model.getAllPackages().keySet()) pkgGraph.put(pkg, new HashSet<>());
336     for (DomainModel.CalculatedElementInfo cls : classes.values()) {
337         String fromPkg = extractPackageName(cls.fullName);
338         if (fromPkg == null) continue;
339         Integer fromScc = classToSccId.get(cls.fullName);
340         for (String dep : cls.dependencies) {
341             String toPkg = extractPackageName(dep);
342             if (toPkg == null || fromPkg.equals(toPkg)) continue;
343             if (isInSameSubtree(fromPkg, toPkg)) continue;
344             if (backEdges.contains(new SCCBreaker.Edge(cls.fullName, dep))) continue;
345             Integer toScc = classToSccId.get(dep);
346             if (fromScc != null && toScc != null && fromScc.equals(toScc)) continue;
347             pkgGraph.get(fromPkg).add(toPkg);
348         }
349     }
350     return pkgGraph;
351 }
352
353 // -----
354 // Step 6
355 // -----
356
357 private void updateDependentRelationships(DomainModel model) {
358     for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
359         for (String dep : cls.dependencies) {
360             DomainModel.CalculatedElementInfo d = model.getClass(dep);
361             if (d != null) d.addDependent(cls.fullName);
362         }
363     }
364     for (DomainModel.CalculatedElementInfo pkg : model.getAllPackages().values()) {
365         for (String dep : pkg.dependencies) {
366             DomainModel.CalculatedElementInfo d = model.getPackage(dep);
367             if (d != null) d.addDependent(pkg.fullName);

```



```

368         }
369     }
370 }
371
372 // -----
373 // Utilities
374 // -----
375
376 private static boolean isInSameSubtree(String a, String b) {
377     return a.startsWith(b + ".") || b.startsWith(a + ".");
378 }
379
380 private static String extractPackageName(String className) {
381     if (className == null || !className.contains(".")) return null;
382     return className.substring(0, className.lastIndexOf('.'));
383 }
384 }

```

Open Points Before Submission

- Confirm Veronika Schwarz as co-author and verify affiliation (e-mail: veronika.schwarz@qaware.de).
- Keep the submission abstract to one page in the INSTICC template.